

面向多智能体基础能力的 可验证 RL 数据构造

基于 AppWorld 与 Harbor 的约束式造题方案

王锦敖

Multi-Agent Task Forge

2026 年 7 月 16 日

摘要 本报告讨论如何把已有的单智能体任务改造成真正需要协作的训练任务，同时保留清晰、可靠、可以批量计算的奖励信号。方案不重新设计任务和判定器，只改变智能体能看到什么、能调用什么，以及信息必须怎样在角色之间流动。任务最终是否完成，仍由 AppWorld 的真实世界状态判定。

目录

1 问题与目标	1
2 能力选择与学习顺序	1
3 从原始任务到协作任务	2
3.1 从真实依赖中选择题目	2
3.2 把单智能体任务改成协作任务	2
4 造题方法：四个要素	3
4.1 不变量：判定器只继承，不设计	3
4.2 算子：候选怎么产生	3
4.3 判据：一个候选算不算数	4
4.4 良率：造十个留几个	5
5 运行环境、轨迹与验证	6
5.1 容器边界	6
5.2 一条任务如何运行	7
5.3 奖励与有效性参照	8
6 难度控制与规模化	9
7 数据样本与实验结果	9
8 讨论与后续延展	10
8.1 先把现有结论验证扎实	10
8.2 从空间约束延展到时间约束	11
9 结语	11

1 问题与目标

一项可用于强化学习的任务，至少要包含三个部分：模型收到的指令、模型可以交互的环境，以及任务结束后给出的奖励。单智能体任务通常比较容易验证，例如检查文件是否生成、数据库是否更新，或者测试是否通过。多智能体任务则更复杂，因为我们不仅关心最终结果，还希望模型学会把子任务交给合适的角色，并在不同角色之间传递必要的信息。

这里很容易出现一个误区：只要增加几个角色和一段群聊，任务就变成了多智能体任务。实际上，如果主智能体自己仍然拥有全部工具，或者任何子智能体都能独立完成任务，那么这些角色就只是形式上的摆设，不会产生新的学习信号。反过来，如果我们直接根据对话是否“像合作”来打分，奖励又会变得主观且难以规模化。

因此，本项目把目标收紧为：先找到已经有可靠判定器的真实任务，再通过权限和信息约束，让协作成为完成任务的必要条件。这样，模型必须展现出协作能力，但奖励仍然只由最终结果决定。

2 能力选择与学习顺序

多智能体系统涉及的能力很多。为了让训练信号尽量清晰，本方案先选择那些能够通过环境结果验证、又能自然逐层组合的能力。MAST 将常见失败模式归纳为系统设计、智能体间失配、任务验证与终止等几类 [4]；MultiAgentBench 也表明 star、chain、tree 和 graph 等通信拓扑会带来不同的协作要求 [3]。但在数据构造阶段，不适合一开始就同时改变所有因素。

表 1: 当前优先训练的能力

能力	模型需要学会什么	如何观察失败
任务拆解与依赖识别	把一个目标拆成若干可执行子任务，并判断哪些中间结果必须先获得	遗漏必要应用、顺序错误、下游缺少上游信息
角色分配	根据子智能体拥有的信息和工具，把子任务交给合适的角色	请求发给错误角色，或要求一个角色访问它看不到的信息
跨角色通信	在委派中补充实体、时间范围和上一步结果，使局部执行服务于全局目标	子智能体完成了自己的请求，但最终世界状态仍然不正确
并行调度	在满足依赖和资源约束的同时减少不必要的串行等待	资源冲突、漏排任务、依赖未满足或 makespan 过长

表 1 中的前三项能力都由当前交付直接考查；并行调度是个例外。它列在这里，是因为它同样属于可由环境状态验证的一类，但本次基于 AppWorld 的交付并不覆盖它：sidecar 是单线程的（AppWorld 的世界本身并非线程安全），委派调用是串行阻塞的，而 $r = N_{\text{pass}}/N$ 这个终态标量无法把 makespan 纳入奖励。要训练它需要一个能表达并发的运行时，不在本次范围内——这与后文不把 Theory of Mind 列为独立维度是同一条理由。

这些能力适合按依赖关系逐步训练。第一层只要求模型识别角色并完成一次正确委派；第二层加入跨应用依赖，要求主智能体把上游结果传给下游；第三层隐藏角色的能力说明，让模型主

动探索“谁可能知道什么”；再往后才加入委派预算、更多角色和更复杂的通信拓扑。这里需要说明一点：第三层其实已经隐含了 Theory of Mind——判断“谁可能知道什么”，正是对其他角色知识状态的建模；表 1 中“跨角色通信”的失败判据也是同一回事，Main 想当然地以为子智能体和自己知道得一样多，于是发出了对方无法执行的委派。之所以仍不把 Theory of Mind 和动态恢复列为独立维度，不是因为它们不重要，而是因为当前 reward 只是一个终态标量，无法把一次失败单独归因到这一环。

建议的进阶顺序为：单次角色路由 $\rightarrow$ star + 能力文档可见 $\rightarrow$ star + 仅角色名称可见 $\rightarrow$ 有限预算与多级依赖 $\rightarrow$ chain/tree 与运行时故障。

当前实现和实验覆盖到第三层。后两层已经有明确的构造方向，但需要在进入训练集前分别证明可解性和难度增量。

3 从原始任务到协作任务

方案可以概括成一句话：保留任务和判定器，只重构智能体访问任务的方式。

原始任务已经包含用户指令、可交互环境和程序化判定器。造题时，我们只在任务外层增加通信拓扑、角色可见性和委派预算等协作约束。无论模型怎样拆解和通信，最后仍由原来的环境测试判断任务是否完成。

更具体地，记原始任务为

$$\tau = (x, \mathcal{E}, J), \quad c = (A, V, T, B), \quad \tau_c = (x_c, \mathcal{E}_c, J),$$

其中 x 是用户指令， $\mathcal{E}$ 是环境， J 是程序化判定器； A 、 V 、 T 、 B 分别表示访问划分、角色可见性、通信拓扑和委派预算。 A 指的是划分的结构——Main 不持有任何 API，每个 specialist 只绑定一个应用；至于具体是哪些应用，那并不是一个调节项，而是由参考解决定（见 §3.1）。 A 是四项中最根本的：没有它，主智能体可以绕过协作直接完成任务，其余三项都无从谈起。约束改变了模型访问任务的方式，但 J 始终保持不变。

这一思路和 SWE-smith 利用既有测试扩展软件任务的原则相近 [2]：尽量复用已经可靠的验证逻辑，而不是为每一道新题重新设计判定逻辑。整个造题过程可以归纳为三步：从真实任务中识别跨应用依赖，根据依赖生成角色集合，再把同一个任务包装成不同的协作配置。

3.1 从真实依赖中选择题目

AppWorld 提供 9 个日常应用、457 个 API 和 750 条复杂任务（当前发布的数据集实际包含 732 条），并使用数据库状态测试判断任务是否完成以及是否造成了额外修改 [1]。本仓库读取 train 和 dev 中带完整 ground truth 的 147 条任务，分析参考解实际调用了哪些应用。

角色集合完全由任务本身决定。例如，一道题的参考解同时调用 phone 和 venmo，那么它自然对应两个 specialist。用于提交答案的 `supervisor.complete_task` 不算一个角色，因为它不提供任何业务信息。按这个规则，147 条任务中有 51 条确实需要至少两个应用，其余的直接舍弃，而不是为了凑出多智能体的形式再硬加无关角色。

3.2 把单智能体任务改成协作任务

改造后的环境包含一个主智能体和若干应用专属的子智能体。主智能体看得到用户目标，但没有任何应用 API；phone specialist 只能访问 phone，venmo specialist 只能访问 venmo。子智能体只知道主智能体发来的局部请求，不知道完整任务。

以当前交付中的一道题为例：“在我的 Venmo 社交动态里，给今天所有涉及室友的交易点赞。”主智能体必须先向 phone 询问哪些联系人是室友，再把姓名列表连同日期范围交给 venmo。任何一个子智能体都不能独立完成整道题：phone 知道人际关系但不能点赞，venmo 可以点赞但不知道谁是室友。由此，任务本身就带着一条清晰的依赖链：

phone 获取室友姓名 → Main 传递姓名与范围 → venmo 遍历并执行操作。

这种构造同时从三个方面考查模型：发现信息掌握在哪个角色手中、按正确顺序组织子任务，以及在委派时携带下游真正需要的上下文。但终态奖励只能判断整体是否完成，无法把一次失败单独归因到其中某一项（见 §8.1）。题面仍然是一个自然语言任务，依赖图只存在于环境和参考解中，不会直接暴露给模型。

4 造题方法：四个要素

SWE-smith 的杠杆不在于 bug 注入有多聪明，而在于它从不设计判定器：把一个本来能跑的仓库弄坏，再让仓库自带的测试判断修复得对不对 [2]。把这套方法拆开来看，只有四个要素：一个不变量（仓库的测试）、一组算子（五种产生候选的方法）、一条判据（补丁打上后原本通过的测试变红，这道题才算数），以及每个算子实测的良率（56%、35%、40.2%、33.8%、96.9%）。

任何一套造题方法都要回答同样的四个问题。本节按这四个要素说明本方案所处的位置，并指出当前的缺口。

4.1 不变量：判定器只继承，不设计

造题可以在两个层面动手：设计世界和判定器，或者只设计约束。两者的失败模式并不对称。选错约束，任务会过易或过难——这是可测量的；判定器设计错了，奖励测的就是错的东西——而这是不可见的，因为你自己的测试会同意你自己的错误。本方案因此始终只动后者：约束改变的是“谁能做什么”，永远碰不到“什么算做完”。这不是图省事，而是用一个可见的失败模式换掉一个不可见的。

需要区分清楚的是：语言模型用来生成没有问题，用来判定才有问题。SWE-smith 的五个算子里有三个用语言模型产生候选，题面也由语言模型生成；但判断一道题成不成立的，永远是仓库自带的测试。

不变量由底座提供，因此底座的选择就是方法的第一步。一个底座可用，需要同时满足四条：自带程序化判定器且其中没有 LLM（不可妥协）；真实、可安装、可执行；至少 train split 带 ground truth，否则只能靠猜哪些任务合格；动作面能沿至少两条角色边界切分。

第五条是本项目在实践中补上的：缝必须是信息缝，而不是操作缝。若两侧可以各做各的、只需排个先后，那是并行分工——不需要跨缝搬运任何事实，此时加角色只是形式。只有当一侧要完成的动作依赖只有另一侧才能取得的事实时，委派与通信才会产生学习信号。这条判据也解释了为什么软件仓库类底座不适合：它们共享同一个文件系统，没有任何事实是私有的，任何分割都只会退化成“把文件贴过去”。

4.2 算子：候选怎么产生

按候选的产生方式，算子分为三类，见表 2。

表 2: 造题算子，按候选的产生方式分类

产生方式	SWE-smith 的对应	本方案的算子	改题/改 玩法	良率	状态
程序化	Procedural (13 种 AST 变换)	partition : 按参考解触及的应用切 角色, Main 不持有任何 API	改玩法	39/51 选 题; 3/3 格	已交付
程序化	Procedural	blind : 隐藏角色能力说明, 只留角色 名	改玩法	0/3 格	已交付
程序化	Procedural	route (委派预算): 限制允许的委派 次数	改玩法	未测	未交付
程序化	Procedural	route (chain/tree 拓扑): 改变谁能 跟谁说话	改玩法	0/3 格	未实现
程序化	Procedural	perturb : 运行中撤下一个角色, 迫 使重规划	改题	—	未实现
语言模型 改写	LM Modify / LM Rewrite	weaken : 抽走指令中的日期与实体, 移入 sidecar 的隐藏用户状态	改题	—	未实现
组合	Combine	combine : 把共享实体的两道题合成 一道	改题, 判 定器需重 构	—	未实现

表中未交付指已经实现但没有进入任何已发布配置, 未实现指尚无代码。委派预算属于前者: 它已在 `partition.py` 中完整实现, 并在 Main 侧与 sidecar 两侧强制执行, 但两个已发布配置传入的都是空预算。chain 拓扑属于后者: 子智能体之间的交接在 `partition.py` 里有定义, 却没有任何调用点; 因此该拓扑下的任务无解, 其难度信号是假的。算子名称是本报告的命名, 代码中对应的是 `Topology`、`Visibility` 与 `delegation_budget` 等参数; SWE-smith 的 PR Mirror 在本方案没有对应, 因为这里没有 PR 可以镜像。

这里有一个必须正视的区别: 改玩法的算子不产生新题, 只产生新配置。无论 `partition` 与 `blind` 怎样组合, 51 条任务仍然是那 51 条; $51 \times 2 = 102$ 是配置数, 而不是题目数。SWE-smith 的 50k 条则全部来自编辑世界本身的算子。若要让题量真正增长, 就必须使用会改动任务的算子——而那正是良率不再免费的地方。

`combine` 是其中唯一需要重构判定器的: AppWorld 的判定器检查副作用, 甲题的“不得新增 venmo 交易”会被乙题的合法操作触发, $J_A \wedge J_B$ 并不是一个自治的判定器。它的潜在产量最大, 却也最接近本方案刻意回避的那类做法。

4.3 判据: 一个候选算不算数

SWE-smith 的判据一句话说得清: 补丁打上, 跑测试, 原本通过的测试变红, 这道题就算数。机械、免费、不含判断。

本方案的判据是一个夹逼。记 do-nothing 下限为 r_0 , 无约束对照为 r_{open} , 某配置的得分为 r , 则该配置只有在

$$r_0 < r < r_{\text{open}}$$

且对照自身不在地板上 ($r_{\text{open}} > r_0$) 时才算数。落到 r_0 ，说明这一格与“什么都不做”不可区分，奖励无法分辨模型是否尝试过，梯度为零——无论原因是任务真的难，还是 harness 坏了；顶到 r_{open} ，说明约束根本没有生效，这一格测的是基础任务而不是协作；而若 r_{open} 自己就在地板上，则基础任务本身已经坏了，任何调节项在它上面的效果都无从测量——在一个已经失败的对照上解读受约束分数，正是把 harness 故障误读成难度的那条路径。

任务可解性是一道独立的闸门，不在这条判据里：它由参考解在容器内实际跑通来证明：由 `appworld_oracle_report.py` 执行，结果记于 `sweep/appworld_oracle.json`。

这条规则此前只停留在文字描述层面，本仓库还曾把 0.333 当作“分割很难”记了两天——它其实是地板。现在它是 `forge/appworld/validity.py`，并可由 `python -m forge.appworld.cli judge` 直接跑出上表。

4.4 良率：造十个留几个

良率指造出的候选中有多少真正可用。SWE-smith 为五个算子各测了一个数，并据此决定把成本投向哪里。造题有两处会筛掉候选，因此有两个良率：**选题**（题库里有多少题合格）和**渲染**（合格的题渲染出的格子有多少能训）。两个此前都没有测过。

选题良率：51 里有 12 条是并列题

`select.py` 的判据是 `len(roster) >= 2`。它证明的是“参考解碰了两个应用”，而它的 docstring 写的是“不能协作的题会被丢弃，而不是被装饰”——这两句话不是一回事，中间那一步从未被验证。

差别就是 §4.1 那条：缝必须是信息缝。若两个应用各做各的、只需排个先后，跨缝不搬运任何事实，加角色只是分工。这是可以测的，而且不需要调用任何模型：解析参考解，做一遍前向污点分析，看是否真有某个应用取得的事实，被另一个应用的动作所消费——出现在它的实参里，或出现在守卫它的条件里。这件事由 `seams.py` 完成，结果记于 `sweep/appworld_seams.json`。

表 3：选题良率：三层过滤

过滤	保留	占比
naive: 把 <code>supervisor.complete_task</code> 也算作协作者	147/147	100%
strict: 剔除提交通道 (<code>select.py</code> 当前的判据)	51/147	34.7%
+ 信息缝: 要求事实真的跨应用流动	39/147	26.5%

51 条里有 12 条没有信息缝，且不是零散分布，而是四个完整题族各三个变体；其中两族与已交付题族同为 `phone+venmo`，却没有缝。给它们切角色，训练的是并行分派，不是协作。

这与 `supervisor` 陷阱是同一形状，只高了一层：100% → 34.7% 那一步剔除的是“不搬运信息的提交通道”，34.7% → 26.5% 剔除的是“不搬运信息的第二个应用”。前一步做过了，后一步没有——代价是把 **23.5% 的装饰题当成了可协作的题**。

值得记一笔的是，这个分析里 15 条题的缝只经守卫子句 (`if ... : continue` 之后的动作)，另 36 条经实参。已交付的三条全属前者。一个只检查“实参里有没有上游返回值”的实现会漏掉它们全部——包括本项目唯一交付过的题族。

渲染良率：合格的题，渲染出的格子能训几个

用 §4.3 的夹逼判据审视 sweep，结果见表 4。

表 4：按夹逼判据重新审视现有 sweep（每格一个 seed）

配置	_1	_2	_3	有效格数	说明
star-docs	持平	持平	0.833	1/3	唯一生效的一格
star-names	持平	地板下	地板下	0/3	两格因如实报告失败而低于地板
chain-names（未实现）	地板	地板	地板下	0/3	已知的假信号

已交付的两个配置共 6 格，只有 1 格落在有用区间内。star-docs 的均值 0.944 来自一格生效、两格与对照持平；star-names 的 0.445 则来自一格与对照持平、两格低于地板（§7 解释了为什么会低于地板）。这些 rollout 的日志都显示 `ran=True` 且无错误，所以它们是真实的失败而非 harness 故障；但对 RL 数据而言结论相同——它们与“什么都不做”之间没有可用的梯度。

这里有一个值得单独记下的观察：这份表在两版 sweep 之间没有变化。上一版 sweep 里 star-names 的均值是 0.778，这一版是 0.445——相差 0.333，即整个量程的三分之一——而“哪些格子能训”的答案一格都没变。均值摆动三分之一而结论纹丝不动，这正是应当逐格判定而不是取均值的理由。

最后必须说明这个数字的适用范围：单 seed 算不出良率。有效性是 rollout 分布的性质，而不是单次抽样的性质；单个 seed 分不清“这一格总在地板上”和“这一格这次只是运气差”。因此上表是对现有抽样的观察，不是良率的估计。把它变成良率需要的是更多 seed，而不是更多配置。

5 运行环境、轨迹与验证

5.1 容器边界

Harbor 将每道题打包为独立任务，包含指令、容器环境、测试和参考解 [5]。主智能体所在的容器只提供一个轻量的 `team` 接口；AppWorld 世界、子智能体和最终状态都位于 sidecar 一侧。这样，主智能体只能通过委派推进任务，不能绕过协作直接操作应用。

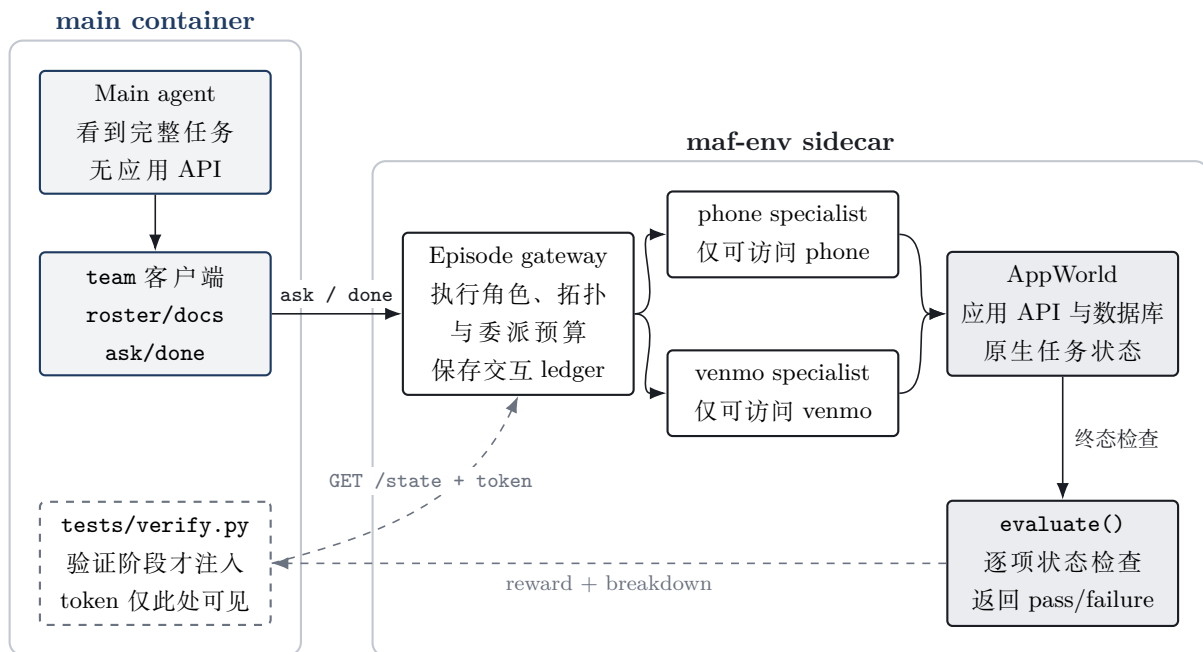


图 1: Harbor 任务的容器架构。左侧实线框是模型可见的 main container；AppWorld、子智能体、ledger 和判定逻辑都留在 sidecar。虚线表示 agent 结束后才发生的验证阶段。

图 1 中最关键的是两条边界。第一，Main 的文件系统里没有 AppWorld、ground truth 或判定器，只能通过 team 发出自然语言委派。第二，verifier token 在 agent 阶段不会出现；Harbor 在执行结束后才把测试注入 main container，由验证器读取 sidecar 中的终态。角色约束因此由环境强制执行，模型也无法在中途读取自己的 reward。

5.2 一条任务如何运行

图 2 展示了任务 2a163ab_1 的参考轨迹。图中每次 team ask 内部都可能包含子智能体的若干轮推理和 API 调用，这里只保留跨角色的信息流。真正决定成败的不是消息数量，而是 Main 是否先获得室友名单，再把名单、日期范围和完整分页要求一起交给 venmo。

用户任务 在我的 Venmo 社交动态里，给今天所有涉及室友的交易点赞。

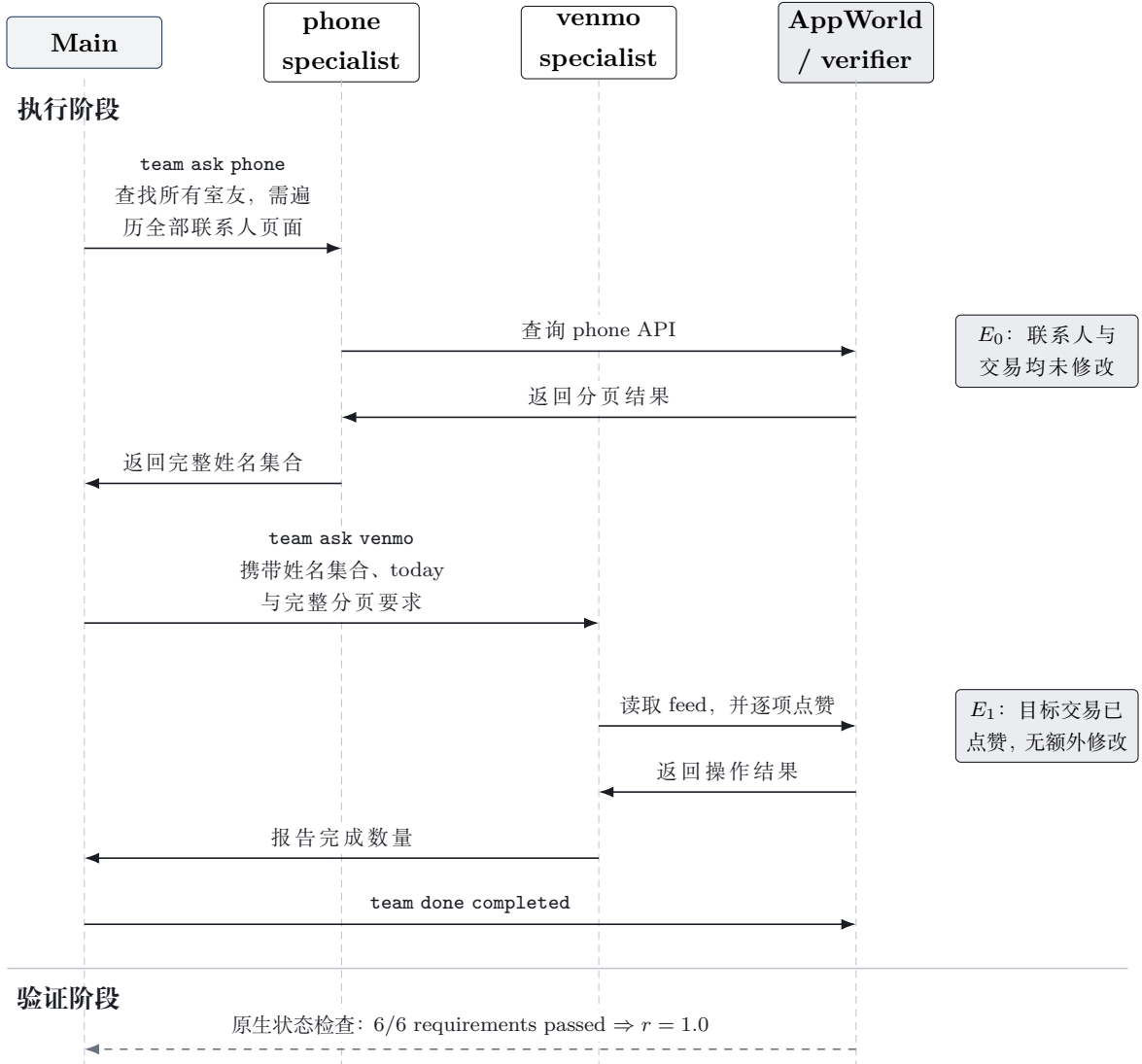


图 2: 参考 trajectory 的信息流。上游角色只负责取得事实，Main 负责把这些事实组织成下游可执行的完整请求；最终奖励来自状态 E_1 ，而不是对轨迹文本的匹配。虚线为 agent 结束后的验证阶段。

5.3 奖励与有效性参照

验证器不评价对话是否“合理”，也不采信模型自己提交的工作记录。若任务共有 N 项状态要求，其中 N_{pass} 项通过，则 reward 为

$$r = \frac{N_{\text{pass}}}{N}.$$

这样既保留了部分完成度，又避免使用 LLM judge。模型可以选择不同的拆解和通信方式，只要最终状态正确且没有额外副作用，就会获得相同奖励。

为了确认 reward 真的有意义，每个配置都需要三个参照。第一个是参考解，用来证明任务可解；第二个是无约束对照，让一个智能体直接拥有全部应用，确认基础任务本身能够完成；第三个是 do-nothing baseline，用来测出不采取任何行动时已经能通过多少检查。只有当约束组位于可解上限和无动作下限之间，并且不同实例产生非退化分布时，这组任务才适合用于 RL。

6 难度控制与规模化

难度主要沿四个方向变化。第一是信息可见性：主智能体可以读取各角色的能力说明，或者只看到角色名称。第二是依赖结构：任务可以从两步顺序依赖扩展到三应用、多分支依赖。第三是通信约束，包括委派预算和 star、chain、tree 等拓扑。第四是任务本身的操作规模，例如需要遍历的页面、涉及的实体数以及状态修改数量。

这些因素需要逐层加入。一个有效的难度调节项应当满足两个条件：参考解仍能稳定通过；与上一层相比，模型的 reward 或失败结构出现可重复的变化。若某个拓扑只是让任务变得不可执行，它产生的是环境故障，而不是更高难度。

从规模上看，当前 51 条跨应用候选任务配合 2 个已实现配置，可以生成 102 个协作配置（仍然是那 51 道题），并为每条任务保留一个无约束对照。环境、任务目标和判定器都可以直接复用，批量生成本身不是主要瓶颈。真正的成本来自子智能体的在线模型调用，以及对每个新约束进行多 seed 验证。因而合理的扩展顺序是：先确认少量配置能否稳定地产生训练信号，再扩大任务族和样本数。

7 数据样本与实验结果

当前选取同一任务族的三个实例作为最小交付样本。三道题都要求先从 phone 获取一类联系人，再让 venmo 对指定日期范围内的相关交易执行操作。它们考查的能力结构相同，但实体、日期和操作规模不同。

表 5: 三条 Harbor 数据样本

Task ID	用户任务	必要协作
2a163ab_1	在 Venmo 社交动态里，给今天所有涉及室友的交易点赞	phone 找出室友姓名， venmo 遍历今天的动态并点赞
2a163ab_2	在 Venmo 社交动态里，给昨天所有涉及兄弟姐妹的交易点赞	phone 找出兄弟姐妹姓名， venmo 遍历昨天的动态并点赞
2a163ab_3	在 Venmo 社交动态里，给昨天或今天所有涉及同事的交易点赞	phone 找出同事姓名， venmo 跨两个日期遍历并点赞

三条样本的 Harbor reference solution 均通过全部 6 项状态检查，reward 为 1.0。随后，我们在相同任务上比较三种配置：open 作为无约束对照；star-docs 要求主智能体委派，但允许查看角色能力；star-names 进一步隐藏能力说明，只保留角色名称。Main 使用 gpt-5.6-sol，specialist 使用 gpt-4.1，每格目前各运行一个 seed。

表 6: 约束消融结果

配置	平均 reward	三条任务	平均耗时/s	含义
open	1.000	1.000/1.000/1.000	30.2	一个智能体拥有全部应用
star-docs	0.944	1.000/1.000/0.833	200.8	强制委派, 可读角色的真实 API 目录
star-names	0.445	1.000/0.167/0.167	145.9	强制委派, 角色能力不可见
do-nothing	0.333	0.333/0.333/0.333	—	不采取行动、但答对答案类型

这张表里两个调节项都不是它们名字所说的东西, 需要逐个拆开。

star-docs 相比 open 只下降 0.056——强模型即使没有直接工具, 也能通过几次委派完成目标。更值得注意的是: 这一档此前的 Main 收到的是一句“它们的能力已列在上方”, 而上方什么都没有, 只有两个角色名; 真正提供目录的是交付容器里的 `team docs`, 而测量走的是另一条进程内路径 (见 §5.1)。这一轮把 2,838 与 5,444 字符的真实 API 目录放到 Main 面前后, 分数一点没变, 仍是 0.944; 变的是委派次数从 2–3 升到 3–5、耗时从 100 秒升到 201 秒。也就是说, 让主智能体看见子智能体能做什么, 增加了委派次数与延迟, 但没有提高成功率。

star-names 的 0.944 → 0.445 看起来终于是可见性在咬人, 其实不是。它三格中有两格得分 0.167——低于 0.333 这条“下限”。它们是这样掉下去的: Main 没能找到目标交易, 于是如实报告“没有找到符合条件的交易”, 而 AppWorld 的 `assert answers match` 期待的是 action 任务的答案 (None), 散文不匹配, 这一项就失分。若 Main 谎称 `completed`, 反而能拿 0.333。

换句话说: 这个奖励为撒谎支付 1/6。诚实报告失败, 比闭嘴装作完成更低分。

因此, 当前数据不足以支持“角色能力未知是更有效的难度来源”这一结论——这不是难度梯度, 而是提交协议与判定器相撞产生的激励。它也说明表中“do-nothing = 0.333”并不是下限: 那是“答对答案类型但什么都不做”的分数, 一次坦诚的失败可以落到它下面。按 §4.4 的夹逼判据, star-names 的有效格数是 0/3, 仍然低于 star-docs 的 1/3。

8 讨论与后续延展

8.1 先把现有结论验证扎实

当前结果仍然是一个小规模验证。三个实例来自同一任务族, 每个配置只有一个 seed, 因此 0.056 和 0.167 还不能视为稳定效应。下一步首先应在现有任务上增加 seed, 给出差异的估计区间; 随后加入 phone-simple note、file system-spotify 以及三应用任务, 确认结论能够跨任务类型成立。

此外, 当前 reward 能可靠判断最终任务是否完成, 却不能单独区分失败来自拆解、角色选择还是沟通。这也是本阶段没有把 Theory of Mind 和动态恢复列为独立维度的原因。更细粒度的能力归因可以通过成对 probe 或环境侧的故障注入来完成, 但仍应由最终状态验证, 而不是重新依赖对话评分。

当 star-names 的效果在多 seed、多任务族上稳定后, 再加入委派预算和 chain/tree 拓扑会更合理。这样每一级 curriculum 都建立在上一层已经掌握的能力上, 难度变化也更容易解释。

8.2 从空间约束延展到时间约束

当前工作改变的是协作的空间结构：谁拥有工具、谁知道哪些信息、消息可以沿什么拓扑传递。下一阶段可以在这套环境上再加入一条时间轴，让信息不是在任务开始时一次性给全，而是在 Agent 主动询问或任务运行到特定状态后出现。这样就能进一步训练隐藏用户需求识别和用户干预下的动态重规划，而不必放弃现有的终态验证。

一种最小改造是从同一个完整任务生成成对样本。对照组仍给出完整指令；实验组只给出一个有意不完整、但符合真实使用习惯的请求，把联系人范围、日期或兼容偏好保存在 sidecar 的隐藏用户状态中。Main 只有在发现信息缺口并询问用户时，才能获得这些约束。两组任务使用相同的初始世界和最终判定器，因此无需单独评价“问题问得好不好”：如果关键问题没有被问到，Agent 就无法稳定到达正确终态；如果什么都问，又会消耗有限的交互预算。

用户干预可以作为下一层难度。在 Agent 已经完成一部分工作后，模拟用户再补充一个范围变化或新的限制，要求 Main 更新计划、保留仍然有效的结果，并把新约束传给相关角色。这里需要区分两种情况：如果后续补充的信息本来就是初始目标的一部分，只是之前没有说清，那么最终 judge 可以保持不变；如果用户真正改变了目标，则必须为新的终态找到同样可靠的环境判定，不能只根据对话判断是否“适应得好”。

这条时间轴扩展可以表示为

$$\tau_{c,h} = (x_0, \mathcal{E}_c, U_h, \Gamma_h, J),$$

其中 U_h 是只存在于 sidecar 的隐藏用户状态， Γ_h 决定信息什么时候被询问、揭示或修改。仍然要守住当前项目最重要的边界：隐藏状态不能进入 Main 的文件系统，交互记录不能由 Agent 自行伪造，最终奖励继续由可重放的环境状态决定。等成对实验能稳定拉开完整指令、隐藏需求和中途干预三种配置的得分差距后，再把它纳入正式 curriculum。

9 结语

这套方案的关键不是凭空编造新任务，而是从已有真实任务中找到协作依赖，再通过权限和信息分配把这些依赖显式化。主智能体必须规划和委派，子智能体只能完成局部工作，最终奖励仍由 AppWorld 的真实世界状态给出。

因此，配置可以批量生成，验证逻辑可以复用。但**难度是否可以沿这些方向调节，本报告并没有证明**。当前三条样本证明的是：权限划分环境与 Harbor 判定器能够正常运行，参考解可以通过全部 6 项状态检查。除此之外，现有 sweep 只在一个 star-docs 实例上观察到非退化的训练信号；star-names 不构成已确立的难度增量 (§7)，chain 拓扑尚未实现，并行调度不在交付范围内。

接下来该做的因此不是扩大规模，而是把当前这一格做实：为已交付配置补足多 seed，剥离提交协议对奖励的污染，并把交付样本从同一题族扩展到不同的依赖结构。隐藏用户需求与中途干预是有价值的延展方向，但它们应当建立在一个已经被证明能稳定产生训练信号的基础配置之上，而当前还没有这样一个配置。

参考文献

- [1] Harsh Trivedi et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. *ACL 2024*, arXiv:2407.18901. <https://arxiv.org/abs/2407.18901>

- [2] John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling Data for Software Engineering Agents. arXiv:2504.21798, 2025. <https://arxiv.org/abs/2504.21798>
- [3] Kunlun Zhu, Hongyi Du, Zhaochen Hong, et al. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. *ACL 2025*, arXiv:2503.01935. <https://arxiv.org/abs/2503.01935>
- [4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025. <https://arxiv.org/abs/2503.13657>
- [5] Harbor Framework. Task Structure: Creating and Running Tasks for Agentic Environments. Documentation, accessed 2026-07-16. <https://www.harborframework.com/docs/tasks>